

IF 260

Sistem Operasi

Minggu 12

Dr. Ananda Kusuma
e-mail: ananda.kusuma@lecturer.umn.ac.id

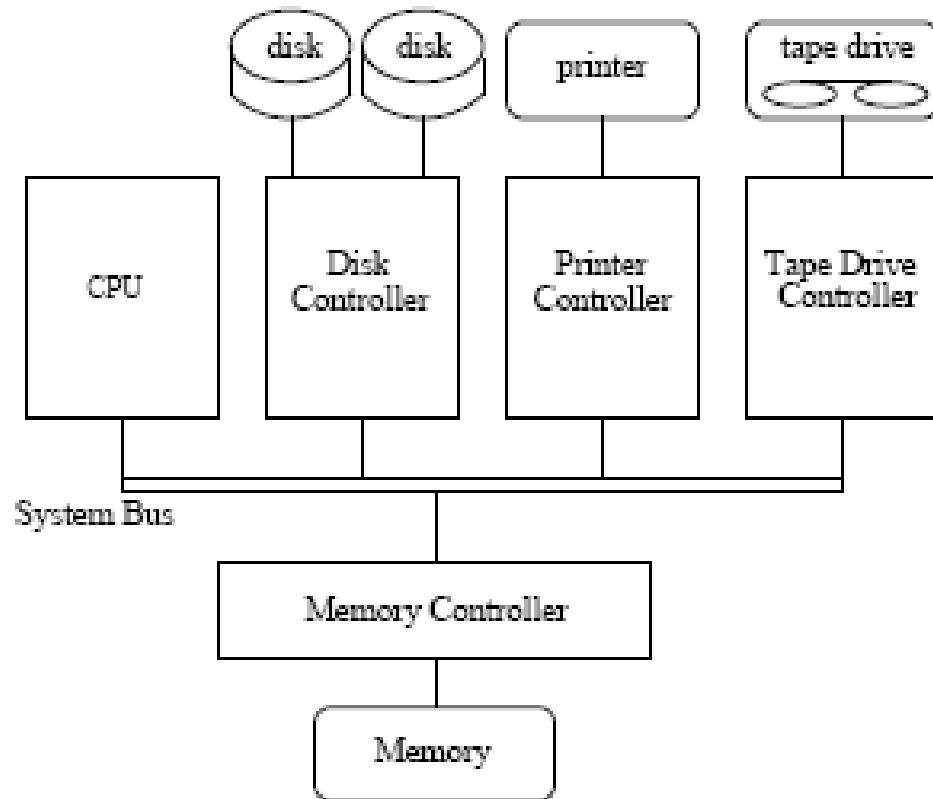
Universitas Multimedia Nusantara
Serpong, Tangerang

Agenda

- **Topik Minggu 12:**
 - **Konsep Input/Output (I/O)**
 - **Prinsip I/O pada Hardware dan Software**
 - **Layer I/O pada Software**
 - **Disks (RAID), Clocks**
- **Quiz**

Konsep I/O (Input/Output)

Gambaran Dasar Sistem Komputer (1) dengan I/O



Gambaran Dasar Sistem Komputer (2)

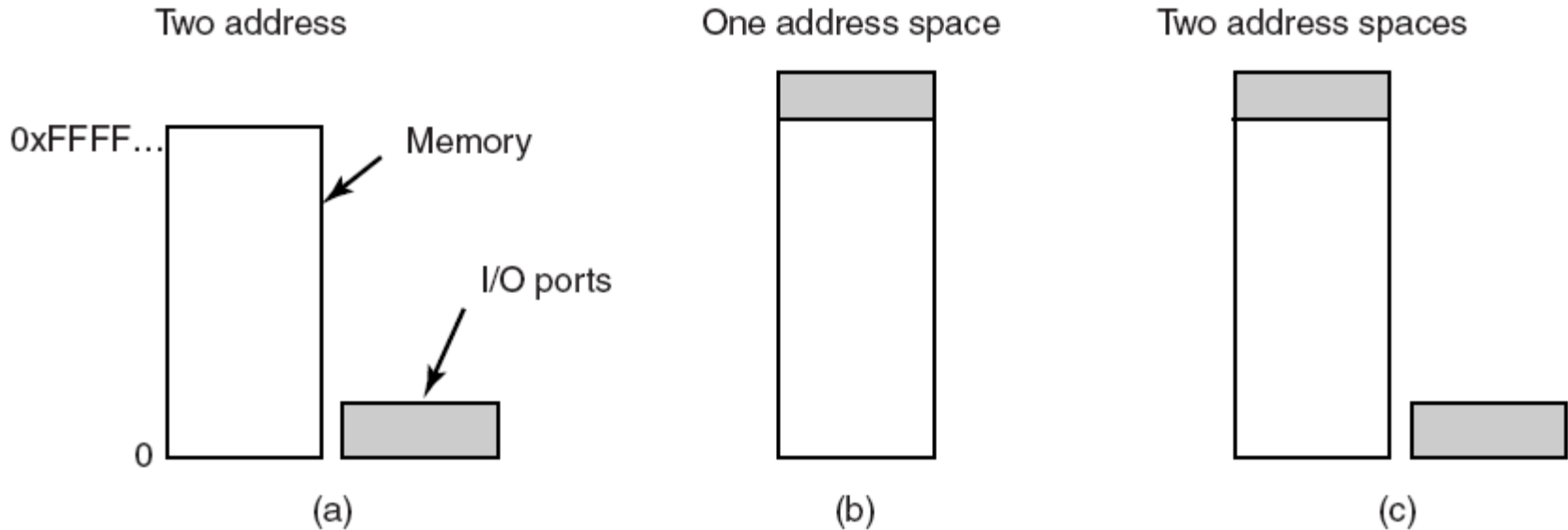
dengan I/O

- **System Bus**
 - Circuit dan protocol yang digunakan bersama-sama oleh devices untuk berkomunikasi dengan CPU
 - Implementasi: PCI bus, ISA bus, dsb.
- **I/O Devices**
 - Contoh: disk drive, printer, scanner, keyboard, mouse, display, clock, dsb.
 - CPU kirim instruksi (IN atau OUT) dan data ke I/O port address untuk pertukaran data antara CPU dan devices
 - Device port memiliki 4 registers:
 - Status: untuk indikasi apakah device busy, data ready atau error
 - Control: command yang hendak dilakukan
 - Data-in: data yang dikirim dari device ke CPU
 - Data-Out: data yang dikirim dari CPU ke device
- Mekanisme komunikasi: polling dan interrupt
- Mekanisme perpindahan data: programmed I/O, interrupt-driven, dan Direct Memory Access (DMA)

I/O device data rate

Device	Data rate
Keyboard	10 bytes/sec
Mouse	100 bytes/sec
56K modem	7 KB/sec
Scanner	400 KB/sec
Digital camcorder	3.5 MB/sec
802.11g Wireless	6.75 MB/sec
52x CD-ROM	7.8 MB/sec
Fast Ethernet	12.5 MB/sec
Compact flash card	40 MB/sec
FireWire (IEEE 1394)	50 MB/sec
USB 2.0	60 MB/sec
SONET OC-12 network	78 MB/sec
SCSI Ultra 2 disk	80 MB/sec
Gigabit Ethernet	125 MB/sec
SATA disk drive	300 MB/sec
Ultrium tape	320 MB/sec
PCI bus	528 MB/sec

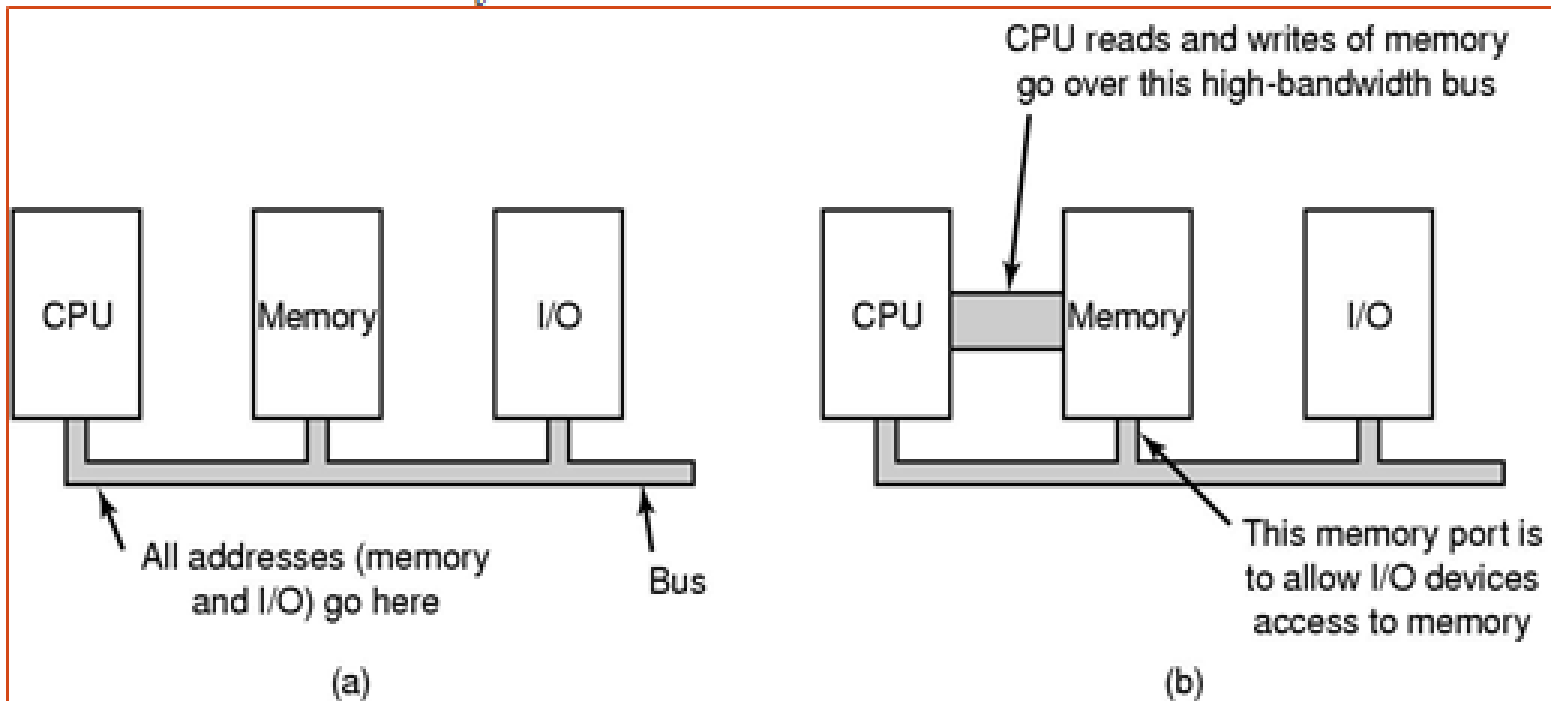
Address Space dan I/O Port Space



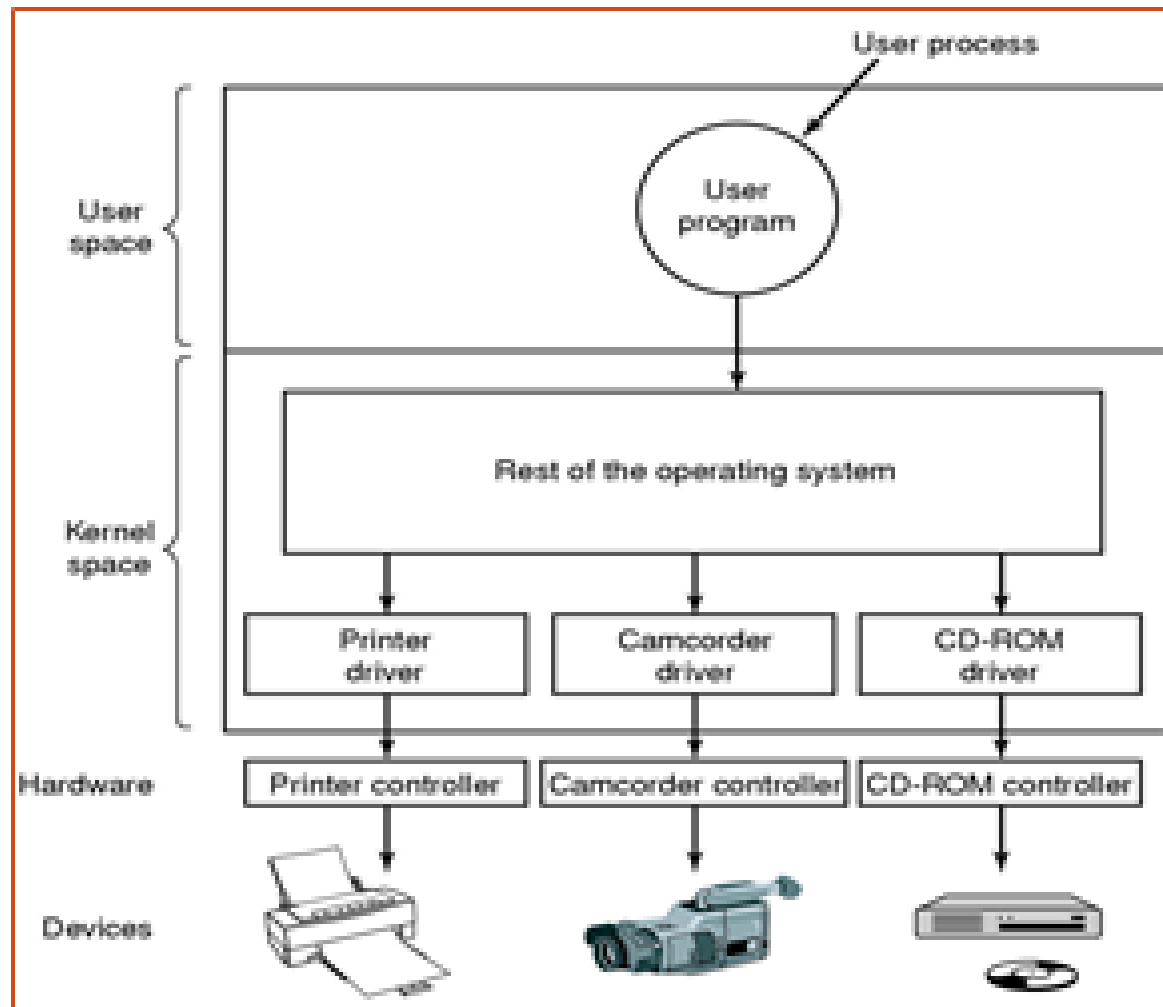
(a) Separate I/O and memory space. (b) Memory-mapped I/O. (c) Hybrid.

Memory-mapped I/O

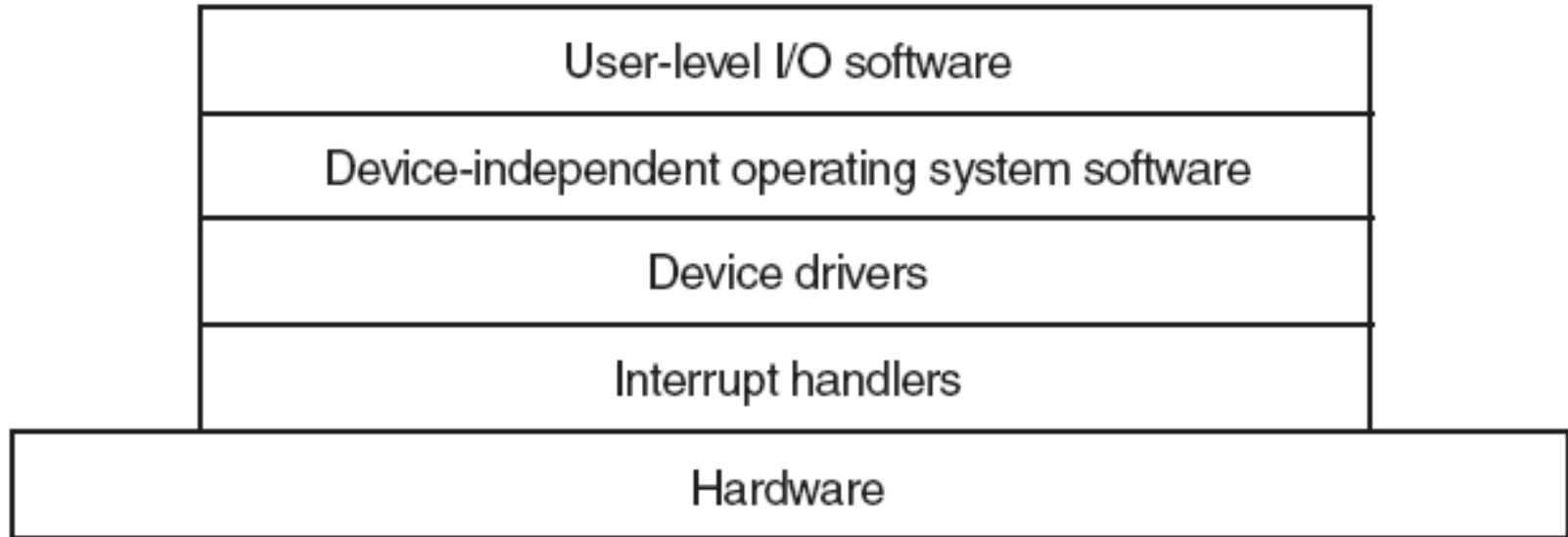
- a) A single-bus architecture
- b) A dual-bus memory architecture



Hardware dan Software I/O



I/O Layers (1)



Hardware: contoh disk untuk media penyimpanan (block device dengan block size bervariasi dari 512 bytes-32KB)

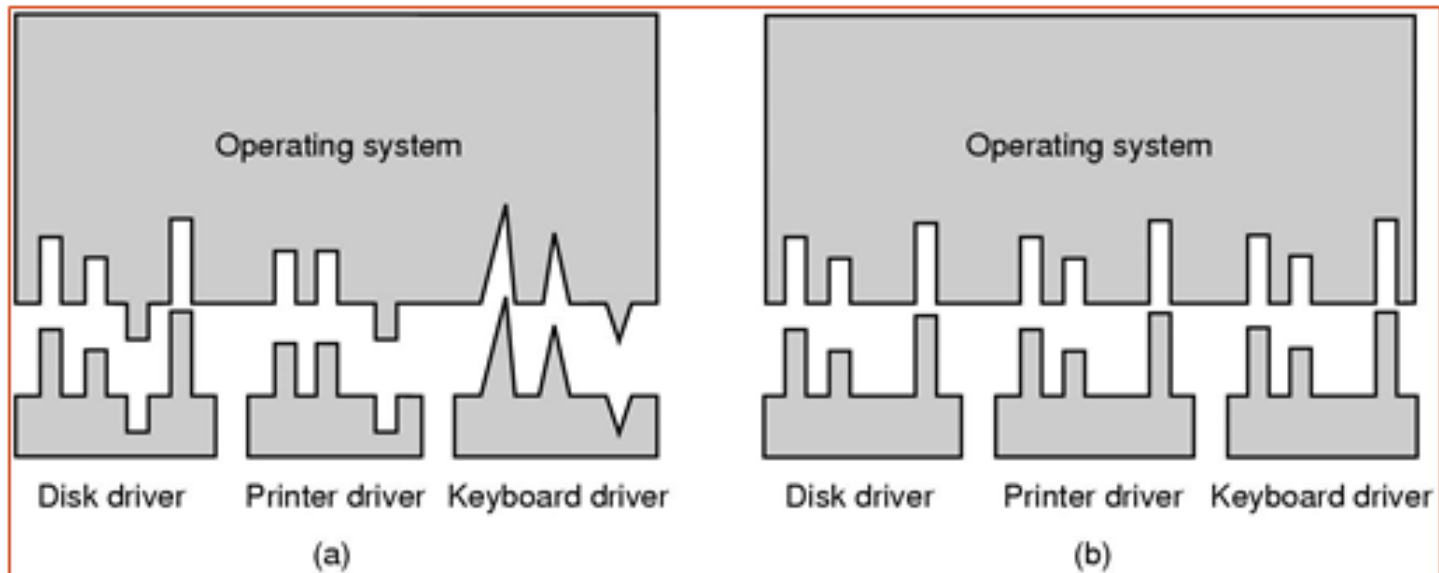
Komponen hardware:

- Mekanikal, baik bersifat magnetic atau optical
- Elektronika → device controller /adapter, secara fisik dapat berupa satu chip, yang bertugas untuk konversi bit streams serial menjadi blok-blok bytes, dan melakukan error correction

I/O Layers (2)

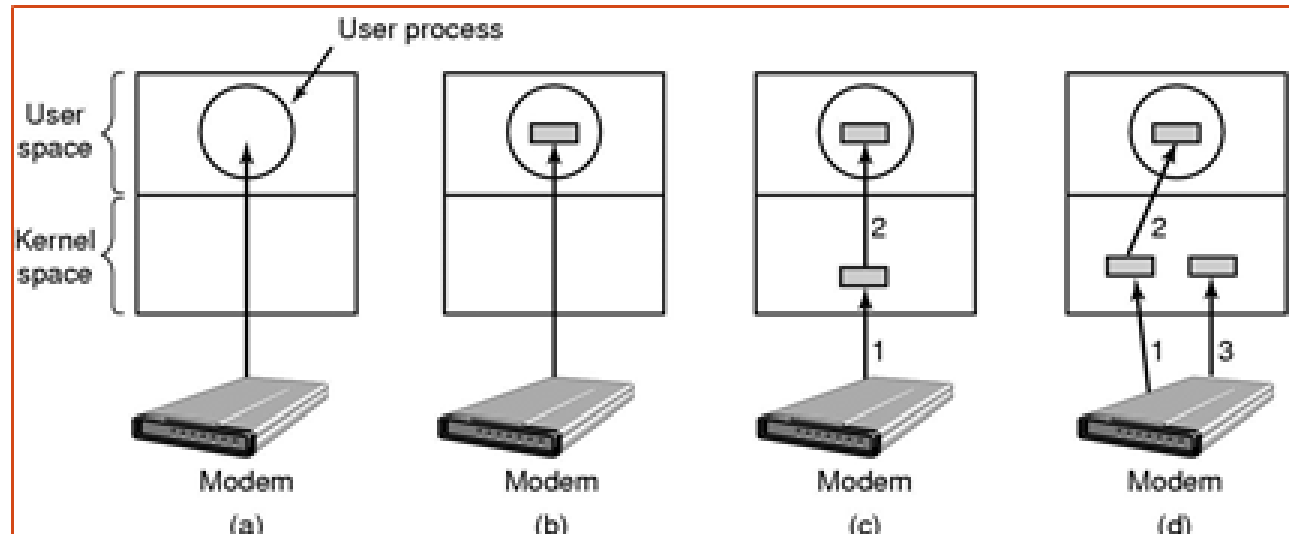
- **Device driver**
 - Code yang spesifik dibuat oleh manufacturer hardware I/O untuk berkomunikasi dengan controller pada hardware
 - Loadable code ke OS kernel
- **Device independent I/O software merupakan bagian dari kernel OS yang melakukan**
 - Uniform interfacing (antarmuka yang seragam) ke device drivers
 - Buffering
 - Error reporting
 - Allocating dan releasing perangkat
 - Penyediaan block size yang independen dengan perangkat

Device Independent I/O Software (1)



- a) Without a standard driver interface
- b) With a standard driver interface

Device Independent I/O Software (2)



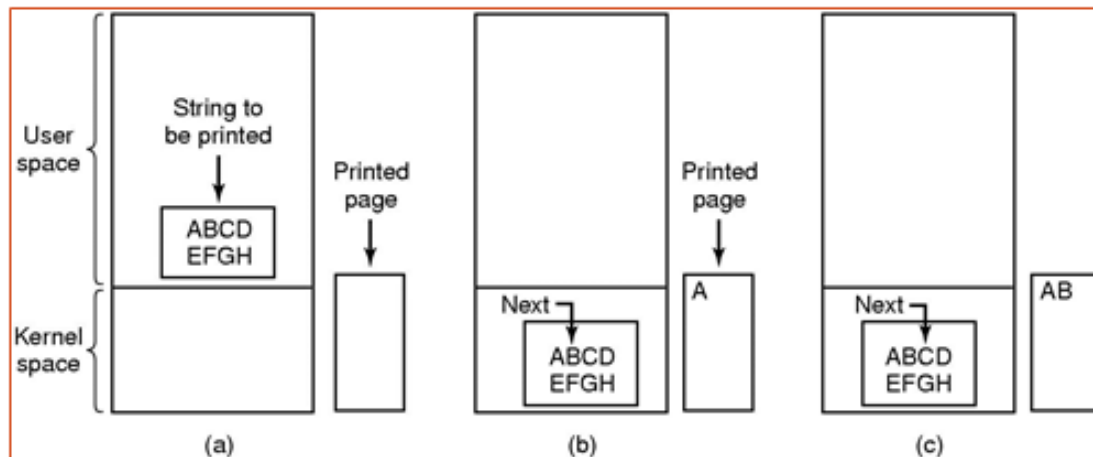
- (a) Unbuffered input
- (b) Buffering in user space
- (c) Buffering in the kernel followed by copying to user space
- (d) Double buffering in the kernel

Polling atau Busy-Waiting

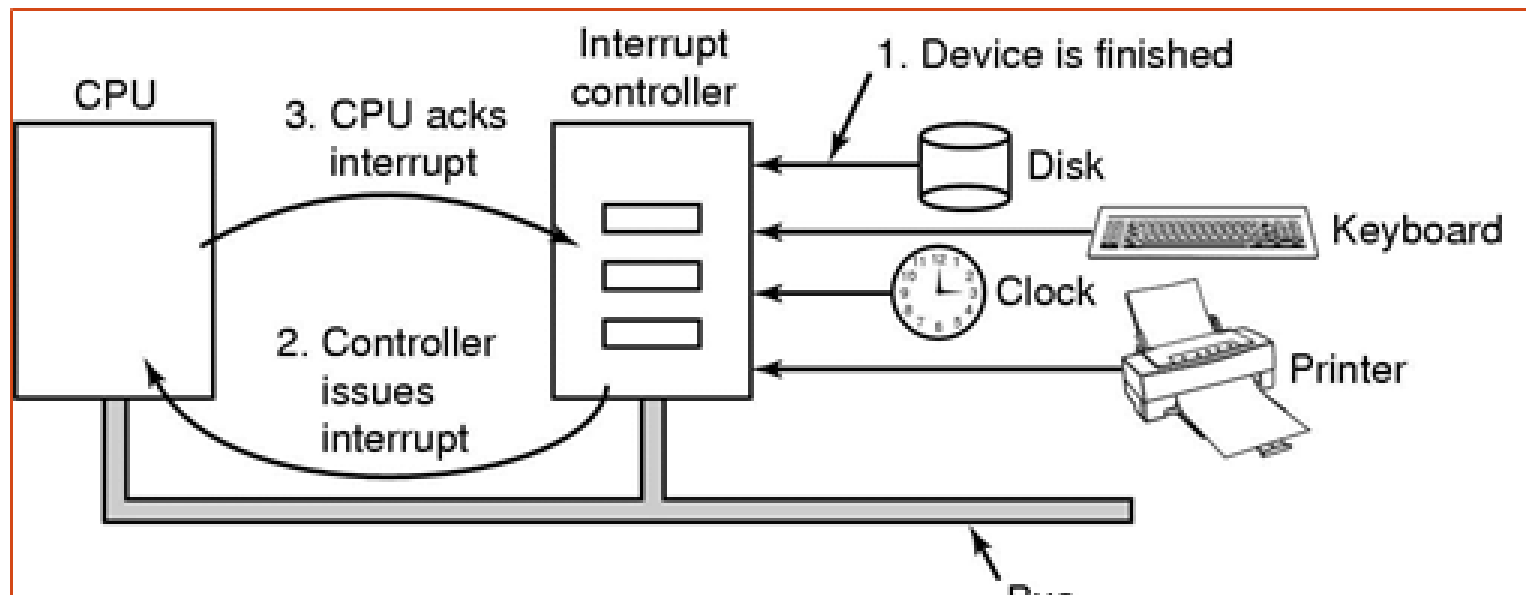
- CPU secara periodik terus menerus memeriksa status I/O devices untuk memeriksa apakah ada data yang akan masuk atau keluar
- Contoh: **Programmed I/O** dengan mekanisme polling:

```
copy_from_user(buffer, p, count);  
for (i = 0; i < count; i++) {  
    while (*printer_status_reg != READY) ;  
    *printer_data_register = p[i];  
}  
return_to_user();
```

/* p is the kernel buffer */
/* loop on every character */
/* loop until ready */
/* output one character */

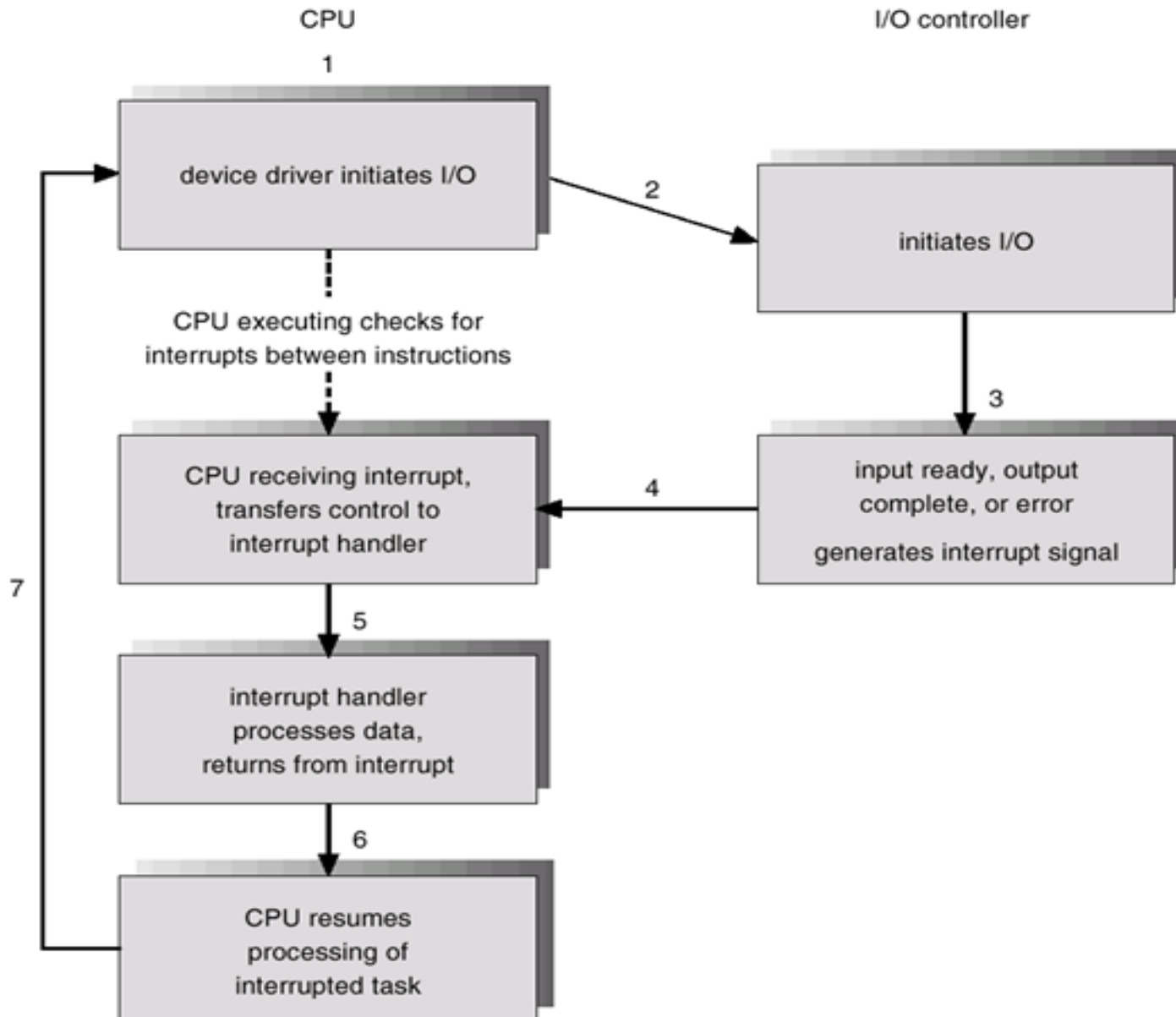


Interrupts Revisited



- How interrupts happens. Connections between devices and interrupt controller actually use interrupt lines on the bus rather than dedicated wires

Interrupt-Driven



Interrupt-Driven I/O

```
copy_from_user(buffer, p, count);  
enable_interrupts( );  
while (*printer_status_reg != READY) ;  
*printer_data_register = p[0];  
scheduler();
```

(a)

```
if (count == 0) {  
    unblock_user( );  
} else {  
    *printer_data_register = p[i];  
    count = count - 1;  
    i = i + 1;  
}  
acknowledge_interrupt( );  
return_from_interrupt( );
```

(b)

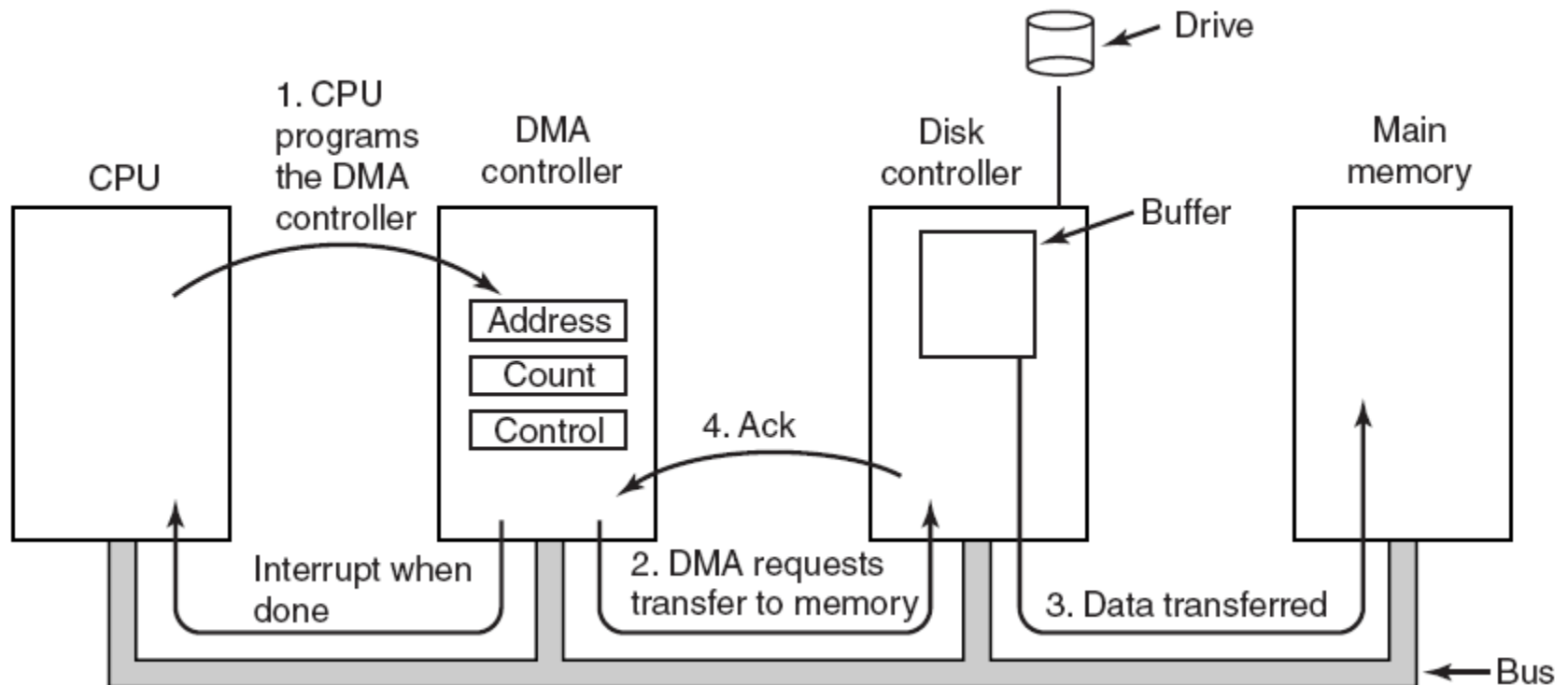
Writing a string to the printer using interrupt-driven I/O.

(a) Code executed at the time the print system call is made.

(b) Interrupt service procedure for the printer.

Direct Memory Access (DMA)

- Metode penanganan I/O di mana device controller langsung berhubungan dengan memory dalam rangka transfer blok data. Peranan CPU hanya di awal, dan saat ada interrupt setelah blok data selesai dipindahkan



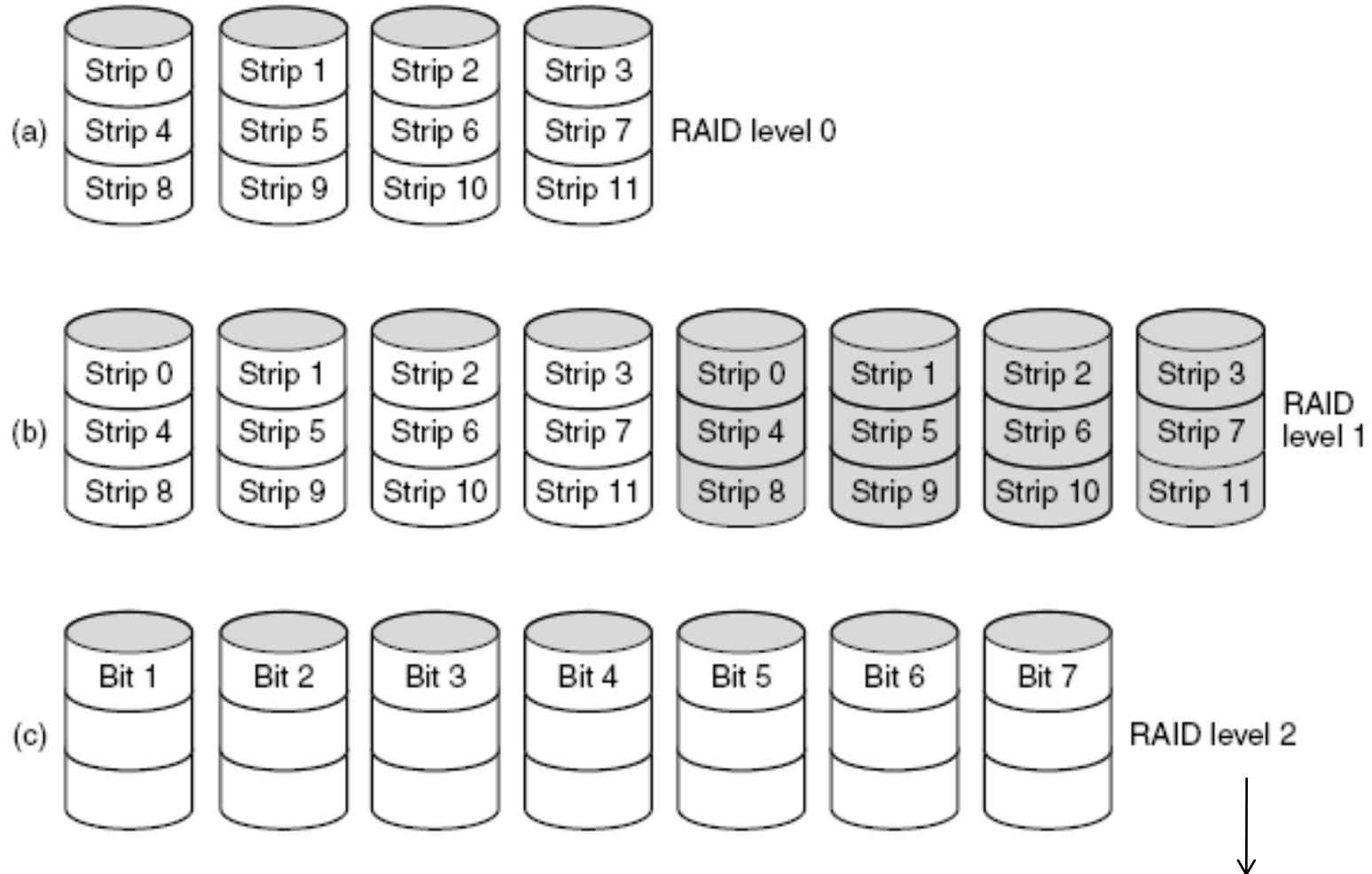
- Contoh:**
- **Disks (RAID)**
 - **Clock**

RAID

- **RAID: Redundant Array of Independent Disks**
- **Penggunaan beberapa disks secara paralel di mana data terdistribusi dengan tujuan meningkatkan**
 - **Performance (kinerja) → dampak paralelisme**
 - **Reliability (kehandalan) → dampak redundansi**
- **Penting untuk server atau data centre**
- **Beberapa disk ini dikendalikan oleh RAID controller yang bertugas mendistribusikan data.**
 - **Hal ini transparan dari sini pengguna**
- **Striping: pendistribusian data ke multiple drives**
 - **Tiap strip pada masing-masing disk terdiri atas sejumlah sector**

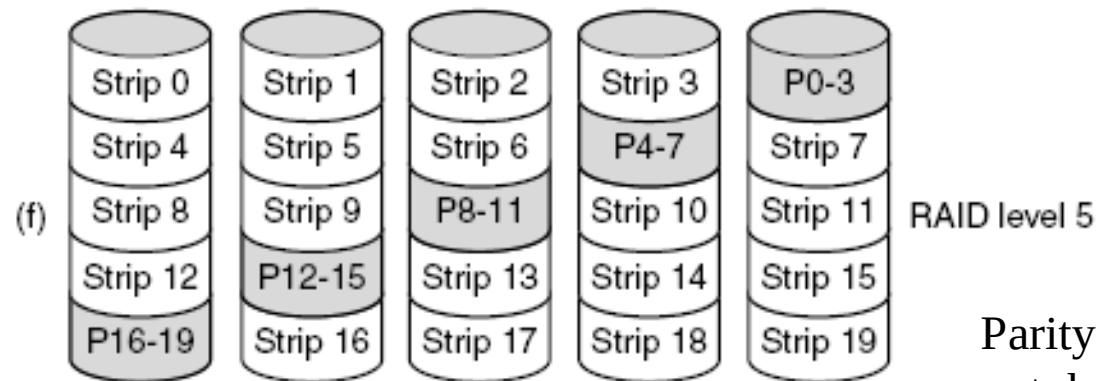
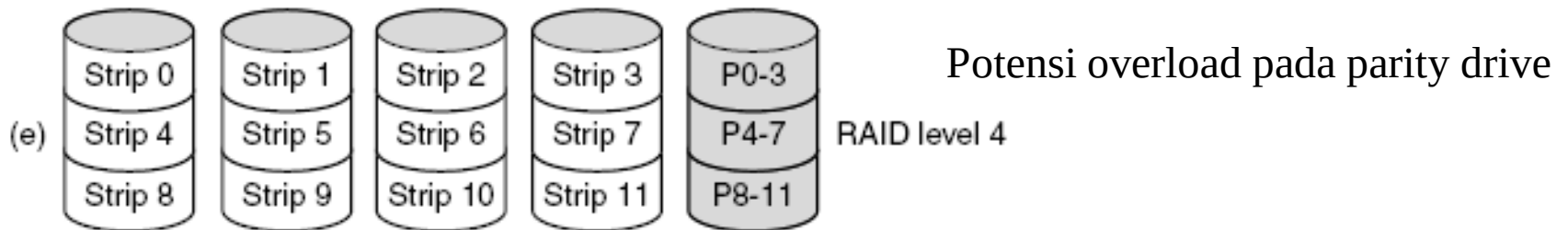
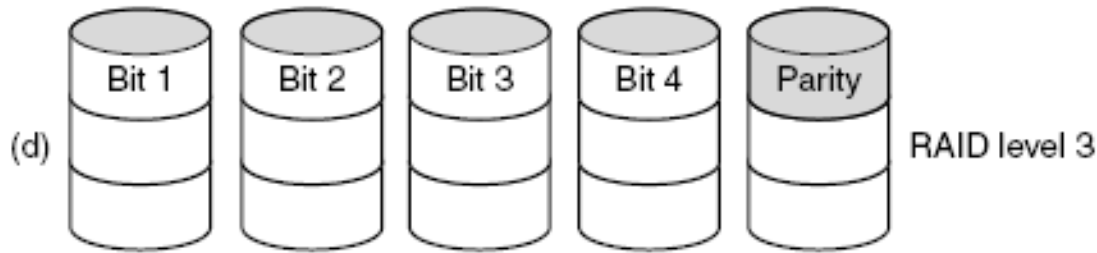
RAID level 0 (Nonredundant), Level 1 (Mirrored) , Level 2 (Memory-Style ECC)

Tiap strip terdiri atas sejumlah sectors



Distribusi dilakukan per bit, maka semua drives harus berputar secara sinkron

RAID Level 3 (Bit-Interleaved Parity), Level 4 (Block-Interleaved Parity), Level 5 (Block-Interleaved Distributed Parity)

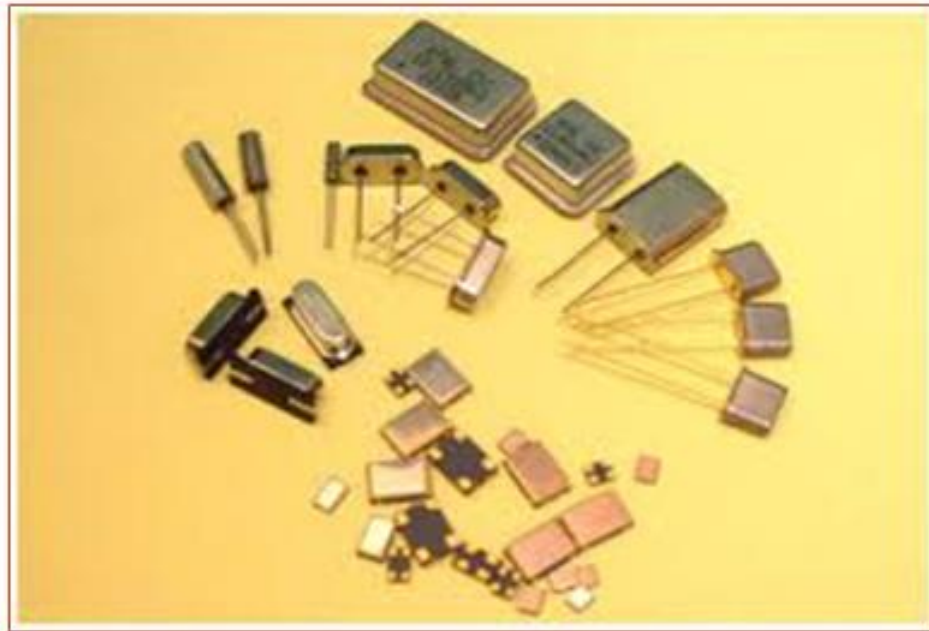


Parity bits didistribusikan secara merata
untuk mengurangi overload

Clock (Timer)

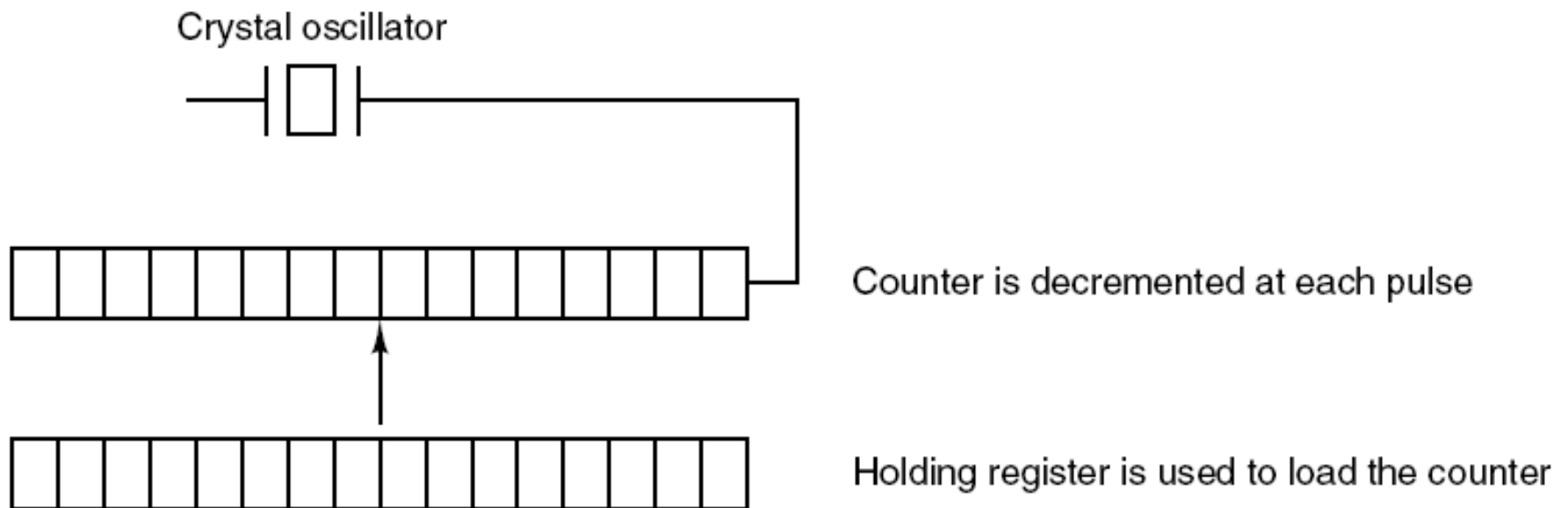
- **Fungsi:**
 - Menentukan real-time clock (waktu saat ini), dan lama waktu yang telah berjalan
 - Menghasilkan clock signal untuk referensi dan sinkronisasi
 - Mengatur durasi CPU untuk tiap process
 - Menentukan waktu alarm untuk dimulainya suatu process
- **Implementasi:**
 - Hardware
 - Pada suatu sistem komputer, ada chipset yang spesifik untuk real-time clock
 - Software

Crystal Oscillator



Clock Hardware (Programmable Clock)

- **Crystal oscillator: membangkitkan pulsa-pulsa (impuls tegangan listrik) periodik yang akurat**
- **Tiap kali ada pulsa, nilai counter diturunkan. Apabila mencapai nol maka CPU di-interrupt**
- **Reprogrammable dengan memasukkan nilai yang disimpan di holding register ke counter**



Programmable Clock Modes of Operation

- **One-Shot mode**
 - Dimulai secara eksplisit oleh software dengan copy nilai holding register ke counter, turunkan nilai counter untuk tiap pulsa, dan interrupt CPU setelah nilai counter sama dengan nol
- **Square-wave mode**
 - Setiap kali nilai counter mencapai nol dan interrupt, nilai holding register secara otomatis di-copy ke counter, dan proses berulang
 - Interrupts yang periodik ini disebut **Clock Ticks**
- **Berbagai mekanisme waktu yang didasarkan clock ticks diatur dengan clock software**
 - Simulasikan virtual clocks
 - Mekanisme soft timers
 - dsb

Programmable Clock: Contoh

- Jika menggunakan 500 MHz crystal, tentukan nilai dari holding register untuk mendapatkan 1 clock tick tiap 1 milliseconds
- 500 MHz crystal artinya tiap detik ada pulsa sebanyak 5×10^8 atau periode tiap pulsa = $1 / 5 \times 10^8 = 2 \times 10^{-9}$ seconds = 2×10^{-6} milliseconds. Untuk mendapatkan 1 clock tiap 1 milliseconds, nilai holding register = $1 / 2 \times 10^{-6} = 50000$

Akhir Kuliah Minggu 12

Terima kasih atas perhatiannya!